

NOME DOS INTEGRANTES E RMs:

Allan Victor Santos de Almeida Jesus (RM573218)

Gustavo Veloso Marchese dos Santos (RM568930)

José Elias Aleixo Lopes (RM568858)

Sarah Mendes Machado De Oliveira (RM570514)

SISTEMA INTELIGENTE DE MONITORAMENTO ESPACIAL

1. Introdução e Contextualização do Problema

A exploração espacial moderna exige que colônias extraterrestres possuam um alto nível de autonomia. Devido à latência (atraso) na comunicação entre a Terra e outros planetas, depender de comandos humanos em tempo real para resolver crises de energia ou falhas de suporte à vida é inviável e perigoso.

O presente projeto tem como objetivo desenvolver o "cérebro digital" da Missão Aurora: um Sistema Inteligente de Monitoramento Operacional. Este software foi programado em Python e tem a capacidade de receber dados brutos de telemetria, organizá-los em estruturas de dados complexas, diagnosticar falhas através de lógica booleana e, de forma autônoma, aplicar técnicas matemáticas de previsão para tomar decisões que garantam a sobrevivência da tripulação.

2. Arquitetura e Estruturas de Dados

Para garantir a eficiência do processamento, a arquitetura de dados abandonou o uso de variáveis simples e adotou estruturas fundamentais da Ciência da Computação, encapsuladas na classe principal `GerenciadorDadosMissao`.

2.1. Dicionários e Hierarquia (Tabela Hash) A escolha técnica para catalogar o status dos sistemas da nave foi o uso de dicionários aninhados. O dicionário atua como uma Tabela Hash, permitindo que a busca pelo status de um módulo específico ocorra em tempo constante, sem a necessidade de varrer listas.

Python

```
1.      # DICIONÁRIO HIERÁRQUICO
2.      self.hierarquia_sistemas = {
3.          "energia": {
4.              "solar": 1,
5.              "baterias": 1
6.          },
```

```

7.         "habitat": {
8.             "oxigenio": 1,
9.             "controle_termico": 1
10.        }
11.    }

```

Através dessa hierarquia, o sistema compreende as dependências da nave, onde os valores booleanos/binários (1 e 0) representam o estado operacional (Operacional e Falha, respectivamente).

2.2. Matriz de Telemetria (Arrays Bidimensionais) Os dados históricos ambientais e de consumo precisavam ser armazenados de forma cronológica. Utilizamos uma matriz (Lista de Listas) onde cada linha representa o estado exato da colônia em um determinado horário.

Python

```

12.     self.telemetria_matriz = [
13.         ["08:00", 30, 20, 80, 15, 0.1],
14.         ["10:00", 55, 25, 85, 18, 0.2],
15.         ["12:00", 70, 30, 90, 22, 0.5], # Pico de radiação
16.         ["14:00", 65, 35, 95, 21, 0.4],
17.         ["16:00", 40, 40, 95, 17, 0.2],
18.         ["18:00", 5, 45, 90, 12, 0.1] # Queda de geração
19.     ]

```

As colunas mapeiam estruturalmente: Horário, Geração Solar (kWh), Consumo (kWh), Bateria (%), Temperatura e Radiação (mSv/h).

2.3. Gestão de Crises: Filas e Pilhas O diferencial do sistema está no tratamento de erros. Implementamos duas estruturas distintas para lidar com os logs e alertas gerados:

- **Fila (FIFO - First-In, First-Out):** Representada por `self.fila_alerts`, processa as ações corretivas. O alerta mais antigo na fila é resolvido primeiro através do comando `pop(0)`.
- **Pilha (LIFO - Last-In, First-Out):** Representada por `self.pilha_logs_criticos`, atua como um sistema de auditoria (Log), onde o erro mais recente é sempre exibido no topo usando o comando `pop()`.

3. Lógica de Negócios e Regras de Diagnóstico

O núcleo de tomada de decisão, localizado no método `processar_regras_e_alertas`, utiliza lógica condicional estrita.

3.1. Expressão Booleana Principal e Níveis de Crise A classificação do status da bateria foi estruturada com uma cadeia de `if`, `elif` e `else`, priorizando as condições de emergência:

Python

```
20. if bateria < 20:
21.     db.fila_alerts.append("CRÍTICO: Bateria em nível de emergência.")
22. elif bateria >= 20 and bateria < 50:
23.     db.fila_alerts.append("ALERTA: Bateria abaixo da metade.")
24. else:
25.     pass
```

3.2. Operadores Lógicos (AND, OR, NOT) e Inconsistências O sistema é imune a falhas isoladas de hardware. Criamos um cenário proposital de inconsistência de dados: o sensor mestre de energia reporta "Normal" (1), mas a bateria real atinge níveis críticos (12%). Utilizando o operador `AND`, o sistema desmascara a falha de hardware. Utilizamos o `NOT` para atestar quedas de conexão, e o `OR` para consolidar ameaças externas:

Python

```
26. # Regra 1: Checagem de Inconsistência Proposital (AND)
27. if db.modulos_criticos["energia"] == 1 and bateria < 20:
28.     db.fila_alerts.append("CRÍTICO: Conflito de dados!")
29.
30. # Regra 2: Falha Crítica de Comunicação (NOT)
31. if not db.modulos_criticos["comunicacao"]:
32.     db.fila_alerts.append("ALERTA: Link offline.")
33.
34. # Regra 3: Variáveis Ambientais Perigosas (OR)
35. if temp > 40 or radiacao > 0.4:
36.     db.fila_alerts.append("ALERTA: Ambiente externo hostil.")
```

4. Análise Preditiva de Dados

Para prevenir apagões, o software não age apenas retroativamente. Ele antecipa o futuro através do cálculo de Média Móvel Simples, uma técnica de previsão aplicada sem a ajuda de bibliotecas de Machine Learning de terceiros, comprovando o raciocínio matemático da equipe.

Python

```
37. def calcular_previsao_proximo_ciclo(lista_consumo):
38.     ultimos_3 = lista_consumo[-3:]
39.     previsao = sum(ultimos_3) / len(ultimos_3)
```

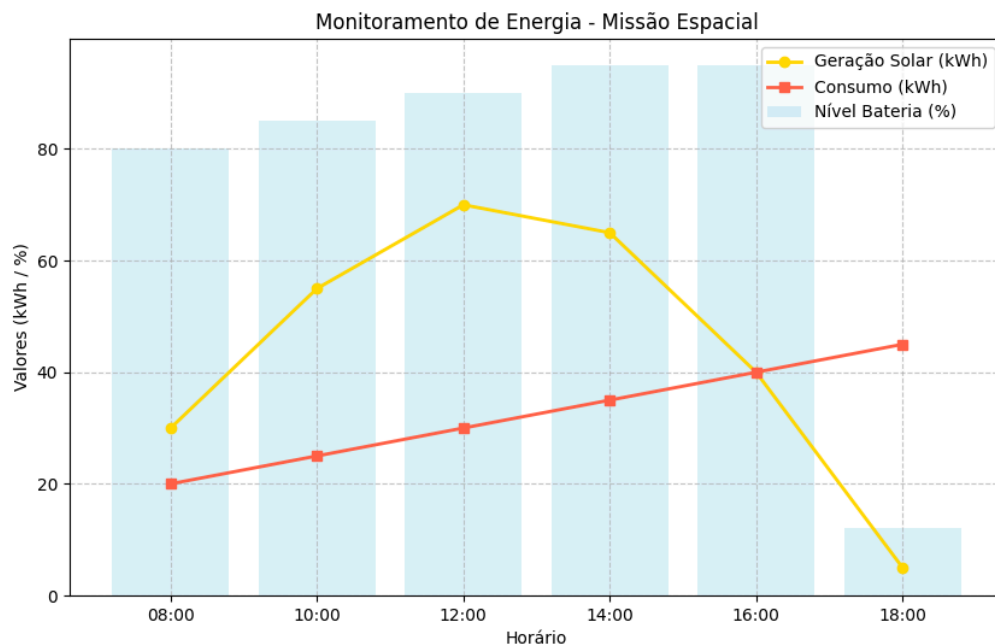
40. return previsao

O algoritmo fatiou a matriz histórica, isolou as três últimas leituras de consumo e dividiu a sua soma. O resultado foi uma previsão precisa de **40.00 kWh** para o próximo ciclo de vida da colônia.

5. Visualização e Tomada de Decisão

Cruzando a previsão de consumo (40 kWh) com a geração atual extraída da matriz (apenas 5 kWh) e a inconsistência da bateria caindo para 12%, o software tomou a decisão técnica de **Desligar o módulo LABORATÓRIO**, poupando recursos vitais para o Suporte à Vida.

Para materializar os dados que sustentam essa decisão, o sistema utilizou as bibliotecas NumPy e Matplotlib para renderizar o seguinte gráfico a partir da matriz de telemetria:



O eixo X do gráfico comprova a passagem do tempo, enquanto as linhas de tendência exibem o cruzamento fatal entre o aumento do consumo (linha vermelha) e a drástica queda da geração solar no fim da tarde (linha amarela), validando matematicamente a decisão do sistema autônomo.

6. Conclusões Finais

O projeto da Missão Aurora comprova que a Engenharia de Software, quando unida ao Pensamento Computacional, é capaz de resolver problemas de altíssima complexidade na indústria aeroespacial.

A escolha pelas estruturas corretas (Filas para execução, Pilhas para auditoria e Dicionários para busca veloz) otimizou o uso de memória. A lógica booleana garantiu que o sistema não fosse enganado por sensores defeituosos. Por fim, a implementação matemática nativa provou que sistemas inteligentes dependem de interpretação de dados para preservar não apenas a continuidade de uma missão, mas o bem mais precioso do universo: a vida humana.